FREE GUIDE + CHECKLIST

Prototype vs. Production

What It Really Takes to Go Live

Your AI tool can generate a working prototype in minutes. It looks real. It feels real. You click around and everything seems to work. So it's ready to launch, right?

Not even close.

A prototype is a proof of concept — it proves your idea can work. A production app is something real people depend on with real data, real money, and real consequences when things break. The distance between those two things is enormous, and it's exactly where most vibe-coded apps fall apart.

This guide walks you through every category of work that separates a prototype from a production-ready product. For each one, you'll get an explanation of why it matters, what can go wrong if you skip it, and a detailed checklist of specific things to verify before you go live.

Some of these you can handle yourself with the right prompts. Others might require bringing in someone with more experience. Knowing the difference is half the battle.

THE GAP AT A GLANCE

PROTOTYPE ("It works on my screen!")	PRODUCTION ("It's ready for real users.")
One user tested it (you)	Handles hundreds or thousands of concurrent users
Works in Chrome on your laptop	Works across browsers, devices, and screen sizes
Hardcoded API keys sitting in the code	All secrets in environment variables, rotated regularly
No login, or a fake hardcoded login	Real auth with hashing, sessions, OAuth, and password resets
Crashes silently when something fails	Graceful errors, user-friendly messages, and error logging
Data in a test database you set up yesterday	Production database with backups, migrations, and access controls
No loading states, no empty states, no offline handling	Every screen handles all 5 states properly

HTTP — data sent in plain text	HTTPS everywhere, all data encrypted in transit
Nobody reviewed the code for security	Security audit completed: injection, access control, data exposure
Console.logs and TODO comments everywhere	Clean code, no debug artifacts, no leaked internal info
"It works" is the only test you ran	Tested on real devices, real networks, with real-ish data
You're the only person who knows how it works	Documentation, README, and deployment instructions exist

1. AUTHENTICATION & ACCESS CONTROL

Authentication is how your app verifies who someone is. Authorization is how it decides what that person is allowed to do. Most security breaches in web apps happen because one or both of these are missing or broken. If your prototype has a login page but no real auth behind it, everything that follows is built on sand.

Why this matters

Without proper authentication, anyone can pretend to be anyone. Without authorization, a logged-in user can access data that doesn't belong to them. These aren't edge cases — they're the most common vulnerability on the internet (OWASP #1: Broken Access Control). AI tools frequently generate code that checks if a user is logged in but never checks if that specific user is allowed to access that specific resource.

> **Real-world example**
>
> Your app lets users view their own orders. The URL is /orders/123. A curious user changes it to /orders/124 and sees someone else's order. Your app checked that they were logged in, but never checked that order 124 belongs to them. This is called an Insecure Direct Object Reference (IDOR) and it's one of the most common bugs in production apps.

Checklist

- [ ] Real authentication is implemented — no hardcoded logins, no fake users, no skipping the login screen
- [ ] Using a trusted auth provider (Supabase Auth, Firebase Auth, Auth0, Clerk) — not a custom-built auth system
- [ ] Passwords are hashed with a strong algorithm (bcrypt, argon2) — never stored as plain text or simple MD5
- [ ] User sessions have a reasonable expiration time — not set to infinite or 30 days for sensitive apps
- [ ] Session tokens are stored securely (HTTP-only cookies, not localStorage visible to JavaScript)
- [ ] OAuth / social login (Google, Apple, GitHub) works end-to-end if offered — tested with real accounts
- [ ] Forgot-password flow exists, actually sends a working reset email, and the reset link expires

- [ ] Email verification is in place for new signups if your app handles sensitive data
- [ ] Every single API route and server action checks: (1) is this user authenticated, and (2) are they authorized to perform this specific action on this specific resource
- [ ] You've tested authorization by trying to access another user's data while logged in — and confirmed it's blocked
- [ ] Rate limiting is enabled on login and signup endpoints to prevent brute-force attacks
- [ ] Failed login attempts don't reveal whether the email exists ("Invalid email or password" not "No account with that email")
- [ ] Admin routes and admin-only features are protected by role-based access control — not just hidden from the UI
- [ ] HTTPS is enabled on every page — all traffic between the browser and your server is encrypted
- [ ] Two-factor authentication is available if your app handles financial data, health data, or other sensitive information

> Prompt to ask your AI tool: "Review every API route in this project. For each one, confirm that it checks both authentication (is the user logged in?) and authorization (is this user allowed to access this specific resource?). Flag any route that is missing either check."

2. SECRETS & ENVIRONMENT VARIABLES

A secret is any value that could cause damage if someone else got it — API keys, database passwords, authentication tokens, webhook secrets, encryption keys. In a prototype, these often end up hardcoded directly in the source code. In production, that's a ticking time bomb.

Why this matters

Bots scan public GitHub repositories 24/7 looking for accidentally committed API keys. When they find one, they can rack up thousands of dollars in charges on your cloud account within minutes. This isn't hypothetical — it happens to developers every single day. The fix is simple (environment variables), but AI tools regularly forget to use them.

> **Real-world example**
>
> A developer pushes code with their OpenAI API key hardcoded in a file. They realize the mistake and delete the key in the next commit. But Git remembers every commit forever. A bot finds the key in the old commit history and uses it to generate thousands of dollars in API calls overnight. The key needed to be rotated (regenerated), not just deleted from the code.

Checklist

- [ ] Every API key, database URL, auth secret, webhook secret, and encryption key is stored in environment variables — zero secrets hardcoded in source code
- [ ] Your .env file is listed in .gitignore and has never been committed to Git (check with: git log --all --full-history -- .env)
- [ ] No secrets exist anywhere in your frontend/client-side code — anything in the browser is visible to every visitor

- [ ] All environment variables are configured on your hosting platform (Vercel, Railway, Render, etc.) — not just in your local .env
- [ ] Different environments (development, staging, production) use different keys and database URLs — never share secrets across environments
- [ ] If any secret was ever accidentally committed to Git (even if deleted in a later commit), that secret has been rotated/regenerated immediately
- [ ] You have not pasted actual API key values into your AI tool's chat — the AI only needs the variable name (e.g., process.env.OPENAI_API_KEY) to write the code
- [ ] Third-party API keys have the minimum permissions necessary — if the key only needs read access, don't give it write access
- [ ] You have a documented list of every secret your app uses, where each one is stored, and when it was last rotated
- [ ] Secrets that are no longer used have been revoked/deleted from the provider's dashboard — not just removed from your code

Prompt to ask your AI tool: "Search the entire codebase for any hardcoded API keys, passwords, tokens, or secrets. Check both the current code and any values that might be embedded in config files, test files, or comments. Also check that .env is in .gitignore."

3. DATA & DATABASE

Your database is the permanent memory of your app. Every user account, every saved item, every transaction lives there. In a prototype, the database is usually a quick setup with no rules, no backups, and test data mixed in. In production, your database is the single most important thing to protect.

Why this matters

Data loss is permanent. If your production database goes down and you don't have backups, everything your users created is gone — and you can't get it back. Beyond backups, an improperly secured database can be accessed directly by attackers, leading to full data breaches. And messy data rules (missing constraints, no cascade deletes) create subtle bugs that compound over time and become nearly impossible to fix once you have real users.

> **Real-world example**
>
> Your app lets users create meal plans that reference recipes. A user deletes a recipe, but the meal plan still points to it. Now the meal plan page crashes because it's trying to load a recipe that doesn't exist. This is a missing cascade delete — and it only shows up in production when real users do unexpected things in unexpected orders.

Checklist

- [ ] You're using a real production database instance — not the development or test database you built the app with
- [ ] Database access is restricted — not publicly accessible, requires authentication, and uses connection pooling
- [ ] Automated backups are configured and running on a schedule (daily minimum for most apps)

- [ ] You've tested restoring from a backup at least once — a backup you can't restore from isn't a backup
- [ ] Row-level security or equivalent access controls are in place — users can only query their own data
- [ ] Required fields are enforced at the database level (not just the frontend form)
- [ ] Unique constraints are in place where needed (e.g., no duplicate email addresses)
- [ ] Cascade deletes are configured: when a parent record is deleted, related records are handled correctly
- [ ] Default values are set for fields that need them (e.g., account status defaults to 'active')
- [ ] Data types are appropriate — dates are stored as dates, numbers as numbers, not everything as text
- [ ] Indexes are added to columns that are frequently searched or filtered (dramatically improves performance)
- [ ] No test data, dummy accounts, or seed data remains in the production database
- [ ] Sensitive data (emails, personal info, payment data) is encrypted at rest if your provider supports it
- [ ] Database migrations are versioned and repeatable — schema changes don't require manual SQL
- [ ] You have a plan for what happens if the database goes down: how long until you notice, how do you restore
- [ ] If your app handles data from users in Europe, you've considered GDPR basics (data deletion requests, consent)

Prompt to ask your AI tool: "Review my database schema. Check that every table has appropriate constraints (required fields, unique constraints, cascade deletes, proper data types, indexes on frequently queried columns). Flag anything that's missing or could cause data integrity issues."

4. ERROR HANDLING & SCREEN STATES

In a prototype, you test the happy path — everything works because you're the one using it and you know exactly what to do. In production, users will lose their internet mid-save, submit empty forms, tap buttons twice, navigate to pages with no data, and do things you never imagined. Every one of those moments needs to be handled.

The 5 states every screen needs

Loading state: What the user sees while data is being fetched. Never show a blank screen — use spinners, skeleton screens, or progress indicators.

Empty state: What the page looks like when there's no data yet. Don't show a blank page — guide the user with a message, illustration, or call-to-action ("No recipes yet — create your first one!").

Error state: What happens when something breaks. Show a human-readable message, not a stack trace. Give the user a way to recover (retry button, go back, contact support).

Success state: Confirmation that an action worked. A toast notification, a checkmark, a redirect — something that says "yes, that worked" so the user isn't left wondering.

Offline state: What happens when the internet drops. At minimum, tell the user they're offline. Ideally, queue their actions and sync when they're back online.

Why AI tools miss this

When you prompt an AI tool to "build a recipe page," it builds the normal state — the page with data, working perfectly. It almost never adds loading spinners, empty state messages, error handling, or offline detection unless you specifically ask. This is one of the biggest gaps between AI-generated code and production-ready code.

Checklist

- [ ] Every screen in your app has been reviewed for all 5 states: loading, empty, error, success, offline
- [ ] Loading indicators appear within 200ms of any action that takes time — no frozen screens
- [ ] Empty states include helpful guidance, not just blank space
- [ ] Error messages are written in plain language — no raw error codes, stack traces, or "500 Internal Server Error"
- [ ] Error messages tell the user what to do next (retry, go back, contact support) — not just what went wrong
- [ ] All console.log statements and debug logging have been removed from production code
- [ ] Verbose error details are logged server-side (for your debugging) but never shown to the user
- [ ] Form validation exists on both the frontend (for user experience) AND the backend (for security)
- [ ] Forms handle: empty required fields, wrong data types, inputs that are too long, special characters, and duplicate submissions
- [ ] Double-click protection on submit buttons — a user can't accidentally create two orders by clicking twice
- [ ] Network timeouts are caught and the user is told what happened (not just an infinite spinner)
- [ ] If an action fails (save, delete, update), the user's data is not lost — the form keeps their input so they can retry
- [ ] Image and file uploads have size limits and type validation — a user can't upload a 500MB file or an executable
- [ ] API errors from third-party services (Stripe, OpenAI, etc.) are caught and handled with user-friendly fallbacks

Prompt to ask your AI tool: "For every page and component in this app, add proper handling for loading state, empty state, error state, and success feedback. Also add a global error boundary that catches unhandled errors and shows a friendly fallback page instead of crashing."

5. HOSTING & DEPLOYMENT

Hosting means your app lives on a server that's connected to the internet 24/7, so anyone can visit it at a real web address. Deployment is the process of getting your code from your computer (or GitHub) onto that server. In a prototype, this might be "it runs on localhost." In production, it needs to be reliable, secure, and recoverable.

Why this matters

A deployment that only works when you manually run commands is fragile. A hosting setup without HTTPS is insecure. An app without a rollback plan means that one bad deployment can take your entire product offline with no way to recover quickly. The goal is to make deployment boring — automated, repeatable, and reversible.

Checklist

- [ ] Your app is deployed to a real hosting platform (Vercel, Netlify, Railway, Render, Fly.io, or equivalent)
- [ ] Deployment happens automatically when you push to your main branch (CI/CD) — not a manual process
- [ ] You've connected a custom domain (yourdomain.com, not just a .vercel.app or .netlify.app URL)
- [ ] HTTPS / SSL is active on your custom domain — the padlock icon shows in the browser
- [ ] Environment variables are configured on the hosting platform — not just in your local .env file
- [ ] You've tested the deployed version thoroughly — not just your local development version
- [ ] The build process completes without warnings or errors (a warning in dev can become a failure in prod)
- [ ] You have a rollback plan: you know how to redeploy the previous working version if the latest deploy breaks
- [ ] Static assets (images, fonts, CSS) are served efficiently — ideally through a CDN for fast global load times
- [ ] Server location is appropriate for your primary user base (a US audience shouldn't hit a server in Singapore)
- [ ] You've tested your app's performance: pages load in under 3 seconds on a normal connection
- [ ] Your hosting plan can handle your expected traffic — if you get featured somewhere, will the site survive?
- [ ] You have monitoring or alerting set up so you know if your app goes down (not just when users complain)
- [ ] Database and backend are on separate services from the frontend if needed (don't deploy your database to a static host)
- [ ] DNS is configured correctly — www and non-www both work, and one redirects to the other

> Prompt to ask your AI tool: "Set up a CI/CD pipeline using GitHub Actions that automatically deploys to [your hosting platform] when code is pushed to the main branch. Include a step that runs tests before deploying, and explain how to roll back if a deployment fails."

6. VERSION CONTROL & CODE QUALITY

Version control (Git) is your safety net. Every change you make is tracked, and you can roll back to any previous version at any time. In a prototype, people often work directly on the main branch with vague commit messages. In production, sloppy version control means you can't figure out what changed when something breaks, and you can't undo it safely.

Checklist

- [ ] All code is in a Git repository hosted on GitHub, GitLab, or equivalent
- [ ] The main branch always contains working, deployable code — broken code lives on feature branches
- [ ] Commits are frequent and have clear messages that describe what changed and why
- [ ] You never commit directly to main for risky changes — use branches and merge after testing
- [ ] .gitignore is properly configured: no .env files, no node_modules, no build artifacts, no OS files (.DS_Store)
- [ ] No secrets have ever been committed to the repository (check the full commit history, not just current files)
- [ ] Large binary files (images, videos, datasets) are not committed to Git — use external storage
- [ ] You know how to roll back to a previous commit if something breaks after a deploy
- [ ] Pull requests (or merge requests) are used for significant changes — even if you're the only developer
- [ ] Old branches are cleaned up after merging — don't accumulate dozens of stale branches
- [ ] A README exists with: what the app does, how to set it up locally, what environment variables are needed
- [ ] Code is reasonably organized: no 1000-line files doing everything, no copy-pasted logic across multiple files
- [ ] No commented-out code blocks left in production — if you don't need it, delete it (Git remembers it anyway)
- [ ] No TODO or FIXME comments that represent actual bugs or missing features — track those as issues instead
- [ ] Console.log, print statements, and debug code have been removed from all production files

7. TESTING & QUALITY ASSURANCE

"I clicked through it and it seemed to work" is not testing. Real testing means systematically verifying that your app works correctly under different conditions, with different data, on different devices, and when things go wrong. You don't need a full QA team, but you do need more than your own gut feeling.

Checklist

- [ ] You've tested every core user flow end-to-end: signup, login, the main feature, editing, deleting, logging out
- [ ] You've tested with realistic data — not just "test" and "asdf" but real names, long text, special characters, and edge cases
- [ ] You've tested on a mobile device (not just your desktop browser's responsive mode — actually open it on a phone)
- [ ] You've tested on at least two different browsers (Chrome + Safari, or Chrome + Firefox)
- [ ] You've tested what happens with a slow internet connection (Chrome DevTools > Network > Slow 3G)
- [ ] You've tested the signup flow with a brand new email that has never used the app before
- [ ] You've tested the forgot-password flow with a real email — confirmed the email arrives and the reset link works

- [ ] You've tested what happens when you submit a form with empty required fields
- [ ] You've tested what happens when you submit a form with extremely long input (500+ characters in a name field)
- [ ] You've tested what happens when you try to access a page that doesn't exist (404 handling)
- [ ] You've tested what happens when you access a protected page while logged out (redirect to login)
- [ ] You've tested what happens when you try to access another user's data via URL manipulation
- [ ] You've tested file uploads with: oversized files, wrong file types, and no file selected
- [ ] You've had at least one other person (not you) use the app without any guidance — and watched where they got confused
- [ ] You've tested the payment flow (if applicable) with a test card in Stripe's test mode or equivalent

8. PERFORMANCE & OPTIMIZATION

A prototype with one user doesn't need to be fast. A production app with real users absolutely does. Slow load times, unoptimized images, and unnecessary API calls don't just frustrate users — they directly hurt conversions, engagement, and search rankings.

Checklist

- [ ] Pages load in under 3 seconds on a regular connection (test with Google Lighthouse or PageSpeed Insights)
- [ ] Images are optimized and properly sized — not loading 4000x3000 photos for a 200px thumbnail
- [ ] Images use modern formats (WebP) where supported and include width/height attributes to prevent layout shift
- [ ] API calls are efficient — you're not fetching an entire database table when you only need 10 items
- [ ] Pagination or infinite scroll is implemented for lists that could grow to hundreds or thousands of items
- [ ] No unnecessary re-renders or duplicate API calls on page load (check the Network tab in DevTools)
- [ ] Static assets (CSS, JS, images) are cached appropriately by the browser
- [ ] Fonts are loaded efficiently — not blocking the initial page render
- [ ] Third-party scripts (analytics, chat widgets, etc.) are loaded asynchronously so they don't slow down the page
- [ ] Database queries have indexes on columns used for filtering and sorting
- [ ] You've tested with a realistic amount of data — not just 3 test items, but 100+ to see if performance holds

9. MAINTENANCE & MONITORING

Launching is not the finish line. Software is a living thing that requires ongoing attention. Dependencies get outdated, security vulnerabilities are discovered, users find bugs you never imagined, and things that worked fine for 50 users start breaking at 500. A maintenance plan is what keeps your app alive after launch day.

Checklist

- [ ] Error monitoring is set up (Sentry, LogRocket, or similar) — you'll know when something breaks before users tell you
- [ ] Uptime monitoring is configured — you get alerted if your app goes completely offline (UptimeRobot, Better Stack, etc.)
- [ ] You have a process for reviewing and updating dependencies regularly (GitHub Dependabot, npm audit, etc.)
- [ ] Security patches for your framework and libraries are applied promptly — not left for months
- [ ] You update one dependency at a time and test between each update — so you know exactly what broke if something does
- [ ] Analytics are in place so you understand how people actually use your app (Plausible, PostHog, or similar)
- [ ] There's a way for users to report issues (support email, feedback form, help page)
- [ ] You have a documented process for what to do when things go wrong: who gets notified, how to roll back, how to communicate to users
- [ ] Database backups are verified periodically — not just configured and forgotten
- [ ] You've thought about scaling: what happens if 10x your current users show up tomorrow? What breaks first?
- [ ] Logs are stored and accessible for debugging — not just console output that disappears when the server restarts
- [ ] You have a recurring calendar reminder (weekly or biweekly) to check error logs, update dependencies, and review user feedback

10. LEGAL & COMPLIANCE BASICS

This isn't legal advice — consult a professional for your specific situation. But these are the basics that every production app should at least consider. Ignoring them doesn't make them go away; it just means you find out about them when something goes wrong.

Checklist

- [ ] Your app has a Privacy Policy that explains what data you collect, how you use it, and who you share it with
- [ ] Your app has Terms of Service that outline what users can and can't do
- [ ] Cookie consent is implemented if your app uses cookies and serves users in Europe (GDPR)
- [ ] If you collect personal data from European users, you have a plan for handling data deletion requests (GDPR right to erasure)
- [ ] If your app targets children under 13, you've reviewed COPPA requirements (US) or equivalent local regulations
- [ ] If you process payments, you're using a PCI-compliant provider (Stripe, PayPal) and never storing card numbers yourself
- [ ] Third-party services you use (analytics, email, hosting) have their own privacy policies and data processing agreements

- [ ] Your AI-generated content (if any) is reviewed for accuracy before being presented to users as factual
- [ ] If users can upload content, you have a basic plan for handling copyright complaints or inappropriate content
- [ ] You've checked that your app name and domain don't infringe on existing trademarks

The bottom line

A prototype proves your idea works. Production proves it works for everyone. You don't need to check every single box before launching — but you should know which boxes you're skipping and why. Some of these you can handle with the right prompts. Others require bringing in someone with more experience. Knowing that difference is one of the most valuable skills you can develop.

Want to learn all of this in depth? Check out Zero to Dangerous: Vibe Coding Fundamentals at teknical.ai